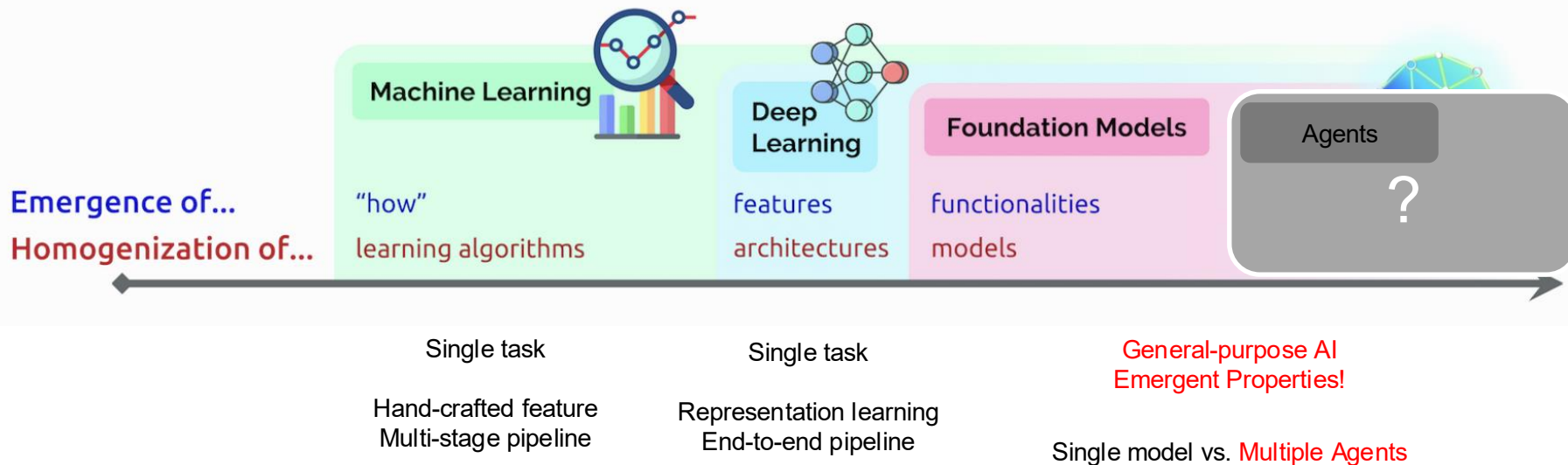


Uninformed Search

Instructor: Hyunwoo J. Kim

Towards AGI



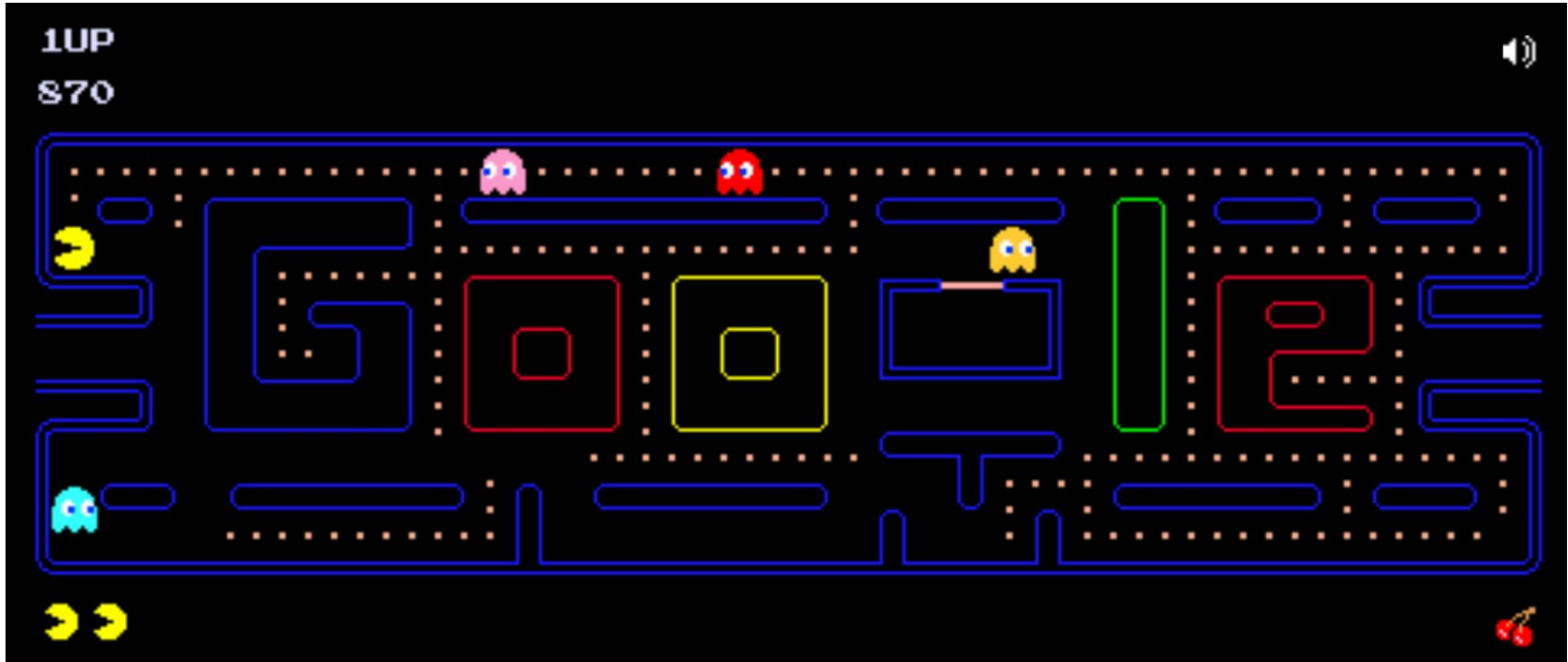
Today's agenda

Attendance Code:

123

- PACMAN!
- Reflex Agents
- Search Problem
- State Space Graphs
- Search Tree
- DFS
- BFS

PACMAN: Let's play it!

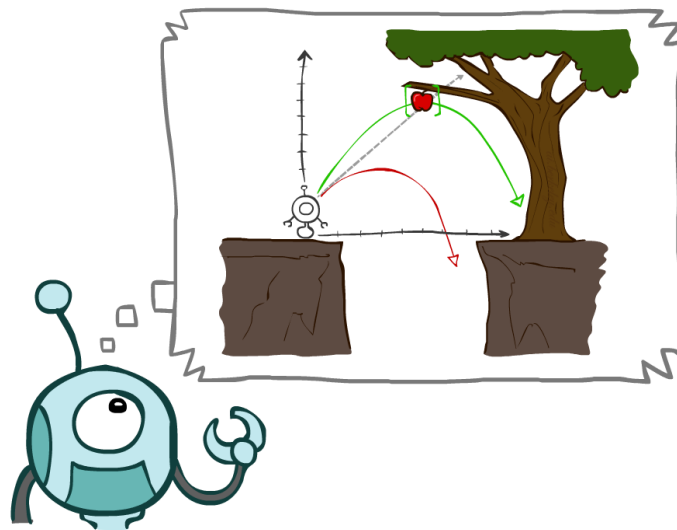


After playing the PACMAN

- Do we have the perfect knowledge of the world?
- Is it easy to do planning?
- What's the best planning?
- Criteria to compare strategies?

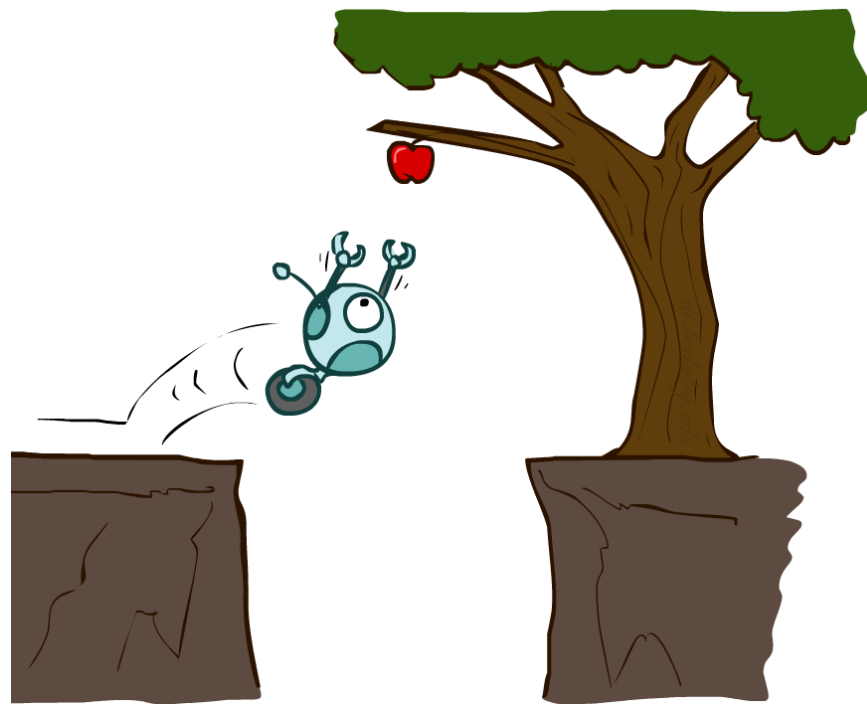
Keywords

- Agents that plan ahead
- Planning ~ Search
- Problem representations
 - State Space Graph
 - Search Trees
- Search Methods
 - DFS
 - BFS
 - Cost-Sensitive Search
 - Uniform Cost search

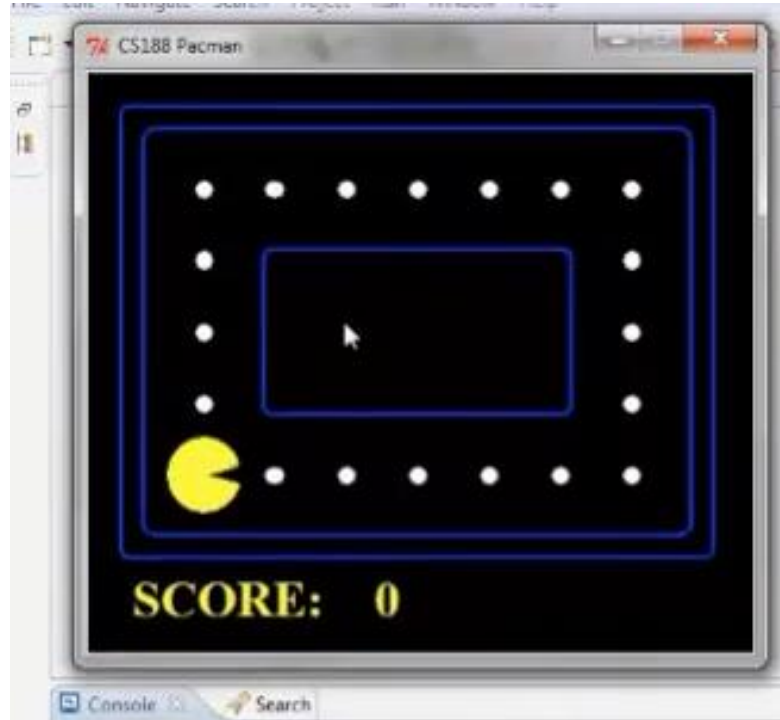


Reflex Agents

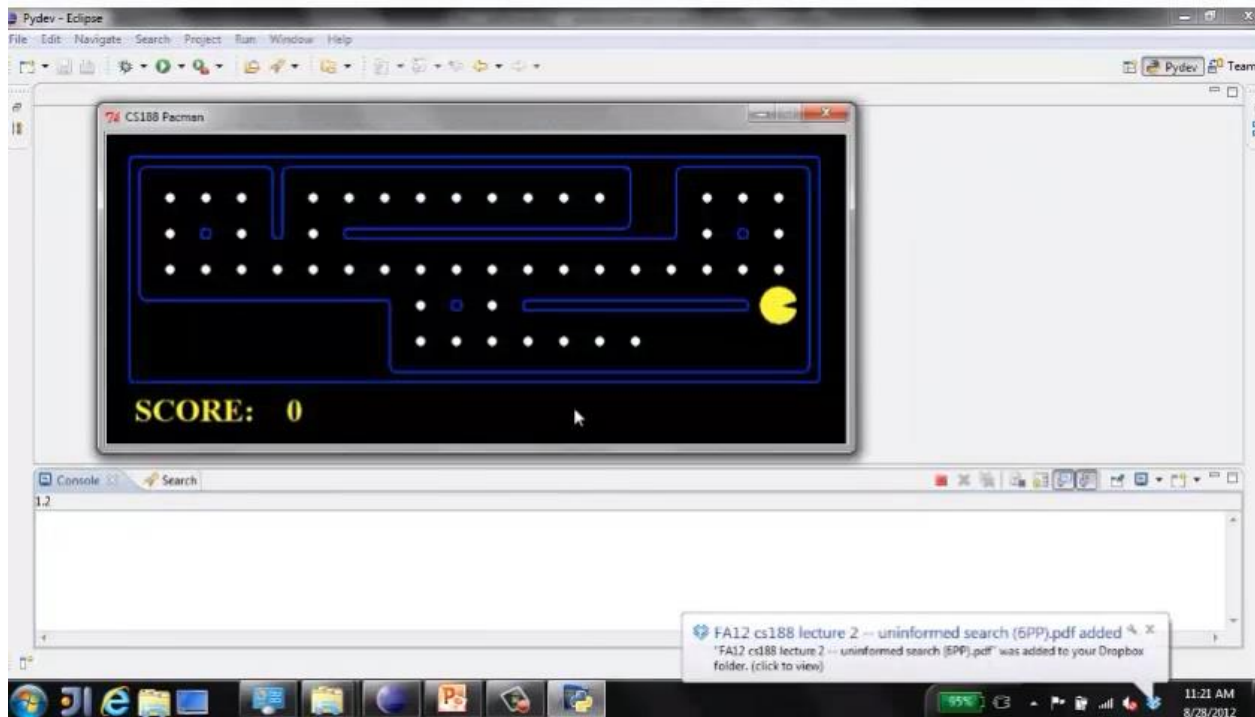
- Reflex agents:
 - Choose action based on current percept (and maybe memory)
 - May have memory or a model of the world's current state
 - Do not consider the future consequences of their actions
 - Consider how the world "IS"
 - Consider how the thermostat works
 - If this, then that
- Can a reflex agent be rational?



Video of Demo Reflex Optimal

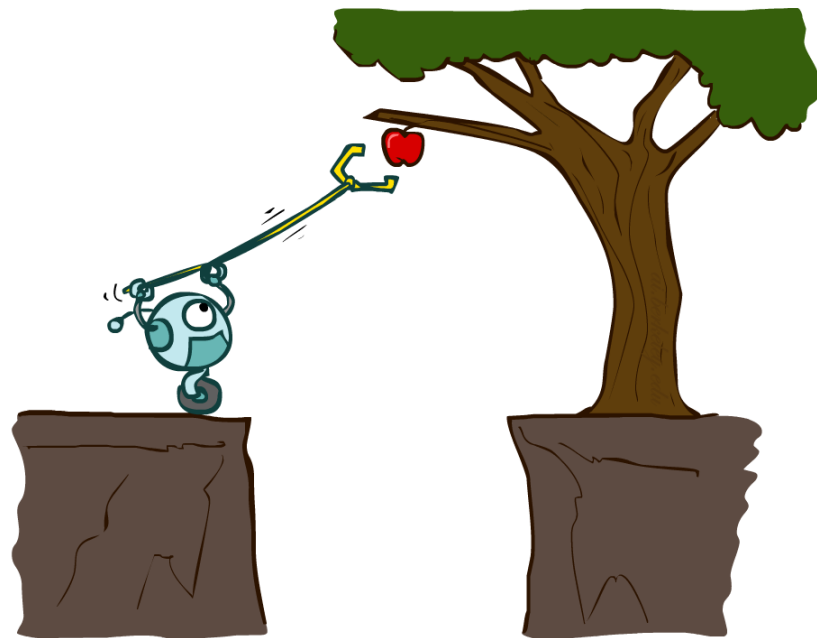


Video of Demo Reflex Odd

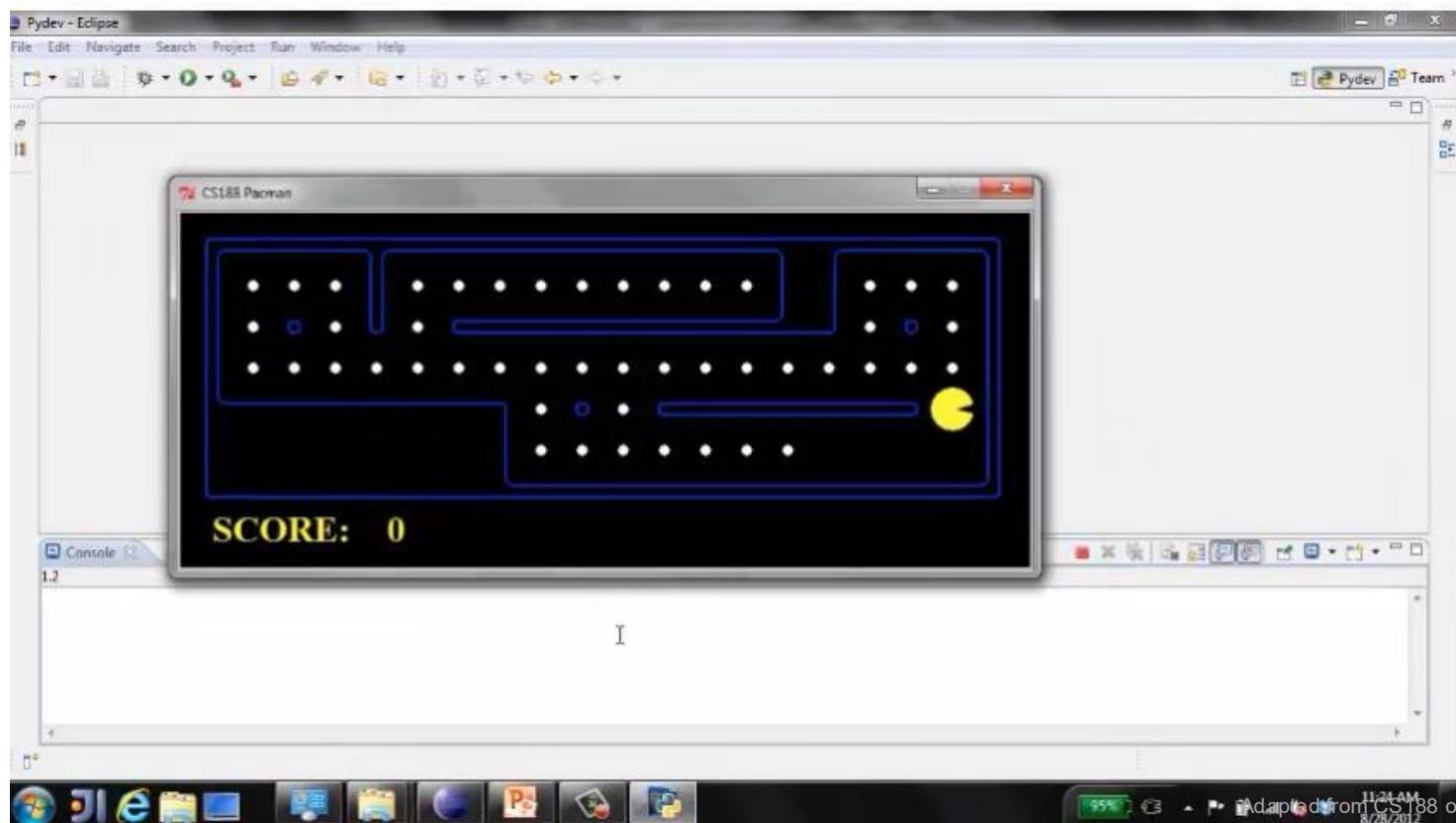


Planning Agents

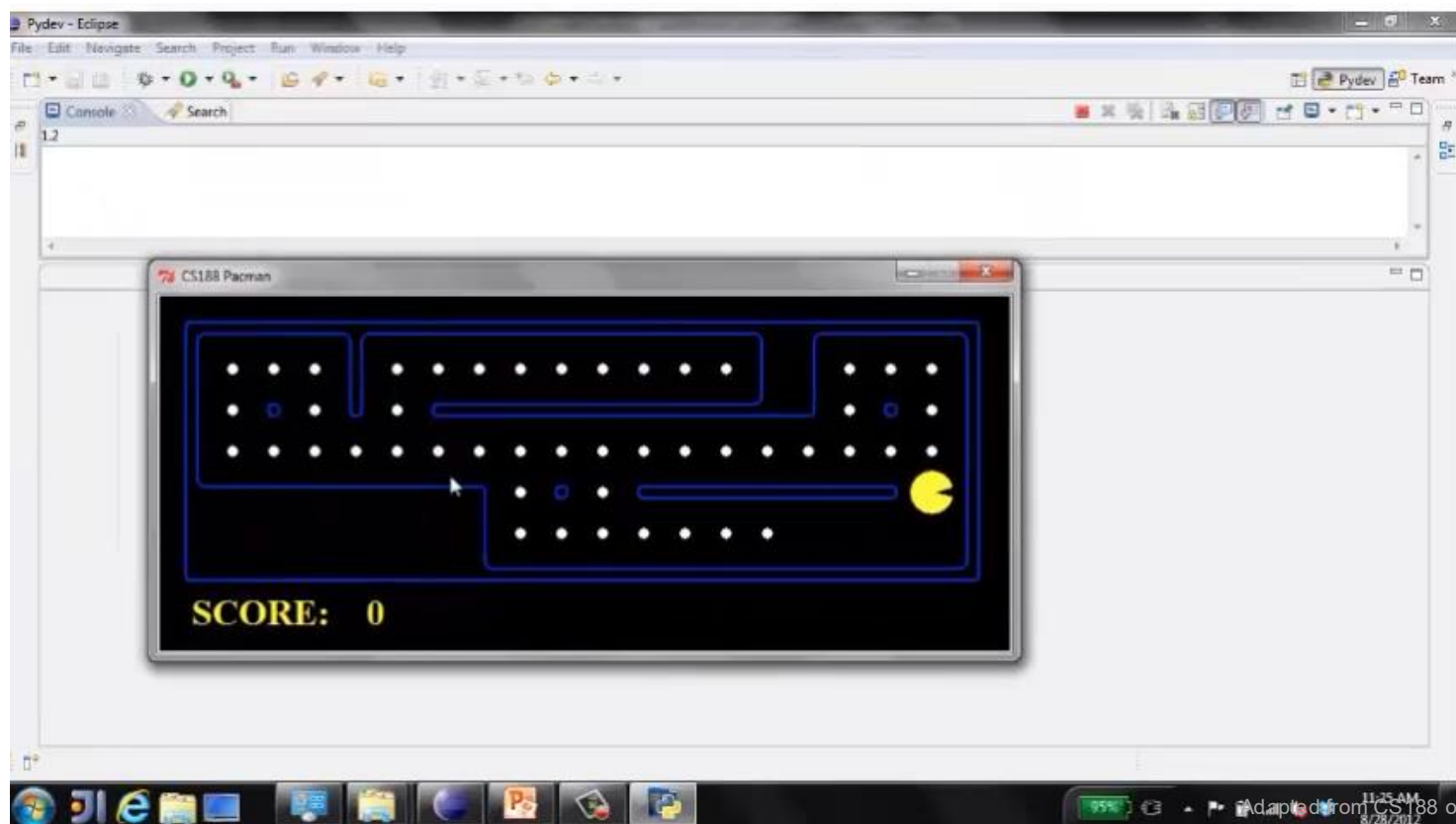
- Planning agents:
 - Ask “what if”
 - Decisions based on (hypothesized) consequences
 - Must have a model of how the world evolves
 - Must formulate a goal (test)
 - Consider how the world “WOULD BE”
- Optimal vs. complete planning
- Planning vs. replanning



Video of Demo Replanning



Video of Demo Mastermind



Search Problems

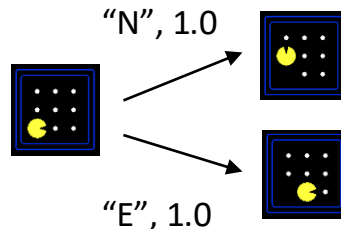


Search Problems

- State space



- Successor function
(with actions, costs)



- A start state and a goal test
- A **solution** is a sequence of actions (a plan) which transforms the start state to a goal state

Search Problems Are Models



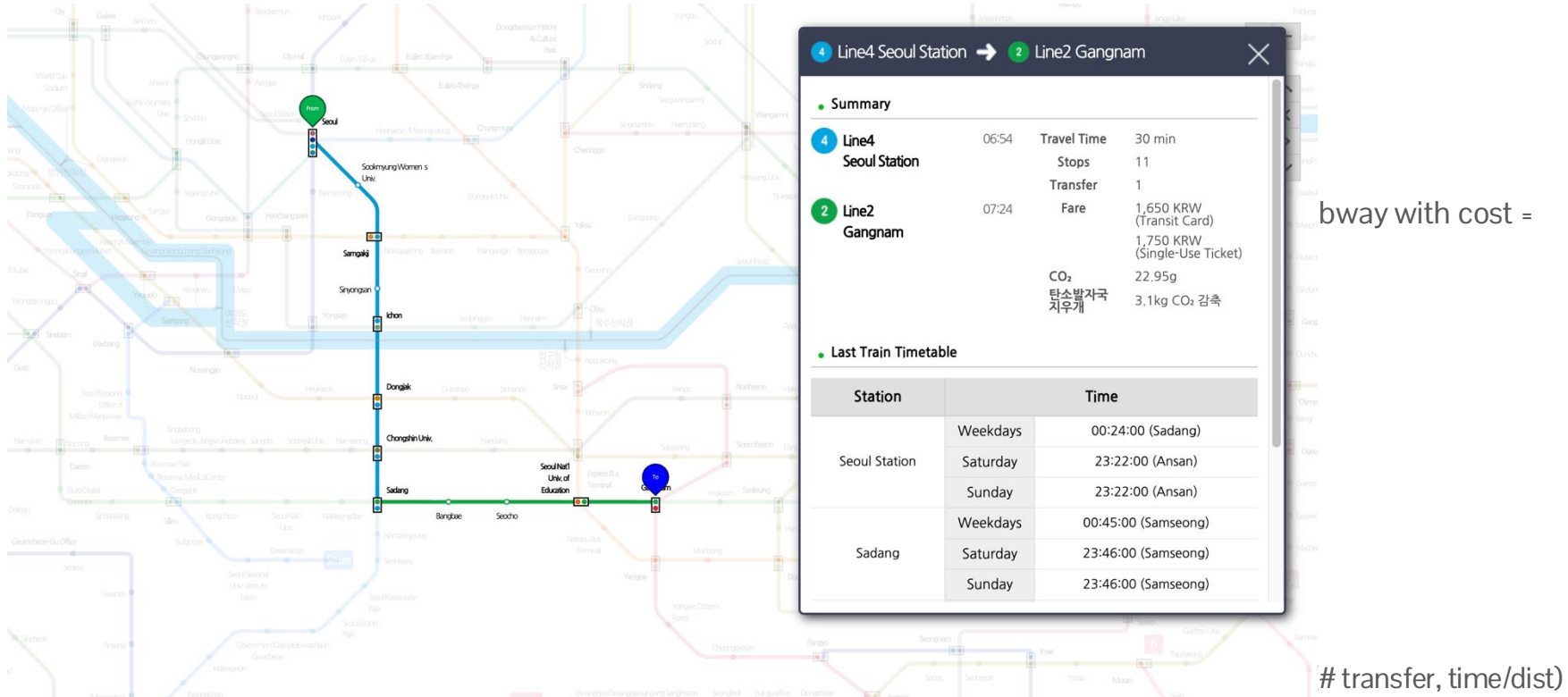
Example: Traveling in Seoul via Subway



- State space:
 - Subway station
- Successor function:
 - Roads: Go to adjacent subway with cost = distance
- Start state:
 - Seoul Station
- Goal test:
 - Is state == Gangnam?
- Solution?
 - Constraints? "Via"
 - Different objectives/cost? (# transfer, time/dist)

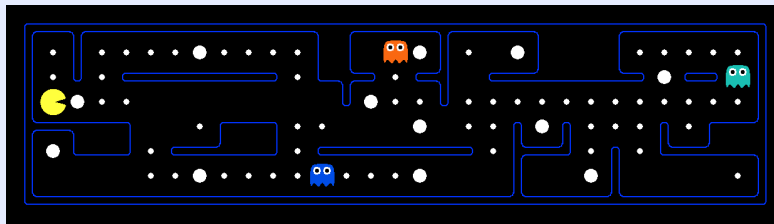
Attendance Code:
123

Example: Traveling in Seoul via Subway



What's in a State Space?

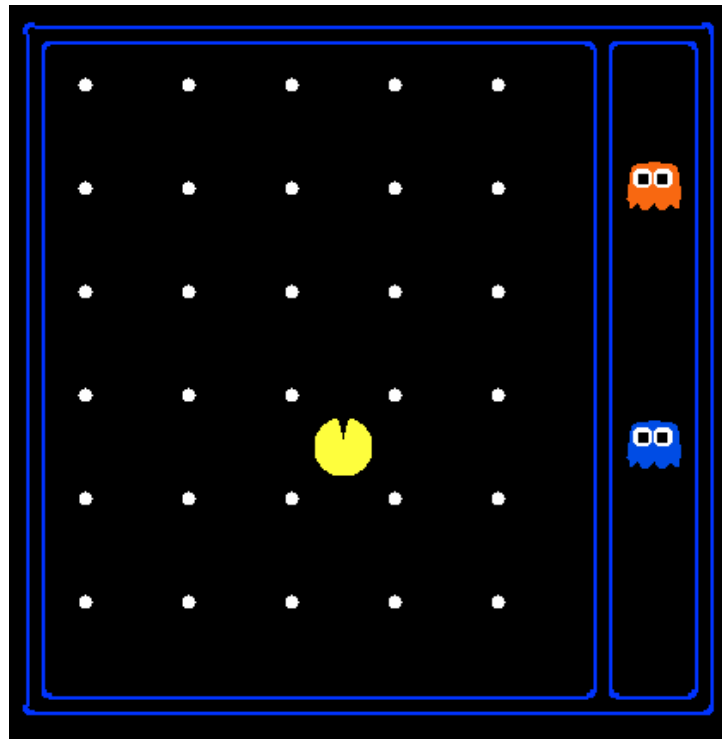
The **world state** includes every last detail of the environment



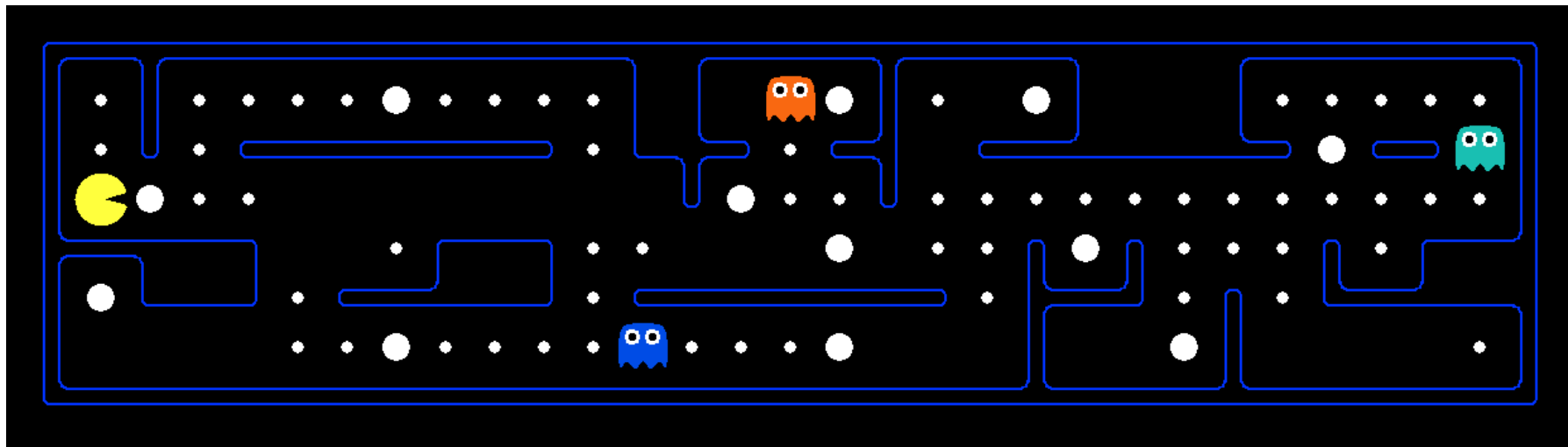
- Problem: Pathing
 - States: (x,y) location
 - Actions: NSEW
 - Successor: update location only
 - Goal test: is $(x,y)=\text{END}$
- Problem: Eat-All-Dots
 - States: $\{(x,y), \text{dot booleans}\}$
 - Actions: NSEW
 - Successor: update location and possibly a dot boolean
 - Goal test: dots all false

State Space Sizes?

- World state:
 - Agent positions: 120
 - Food count: 30
 - Ghost positions: 12
 - Agent facing: NSEW
- How many
 - World states?
 $120 \times (2^{30}) \times (12^2) \times 4$
 - States for pathing?
120
 - States for eat-all-dots?
 $120 \times (2^{30})$

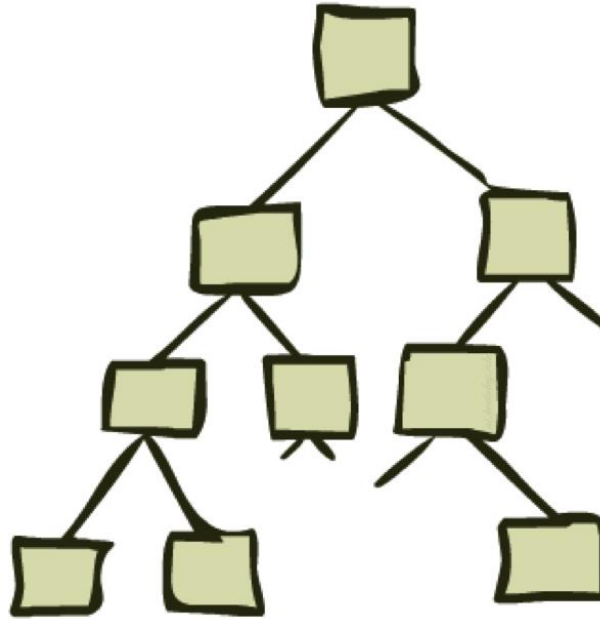


Quiz: Safe Passage



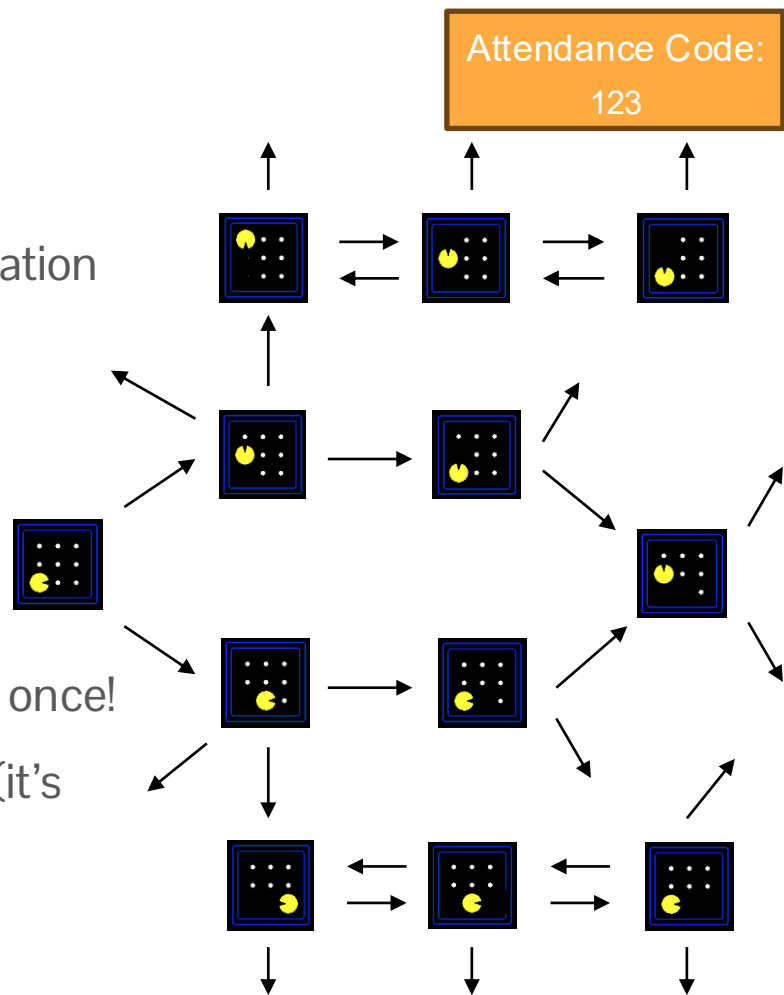
- Problem: eat all dots while keeping the ghosts perma-scared
- What does the state space have to specify?
 - (agent position, dot booleans, power pellet booleans, remaining scared time)

State Space Graphs and Search Trees



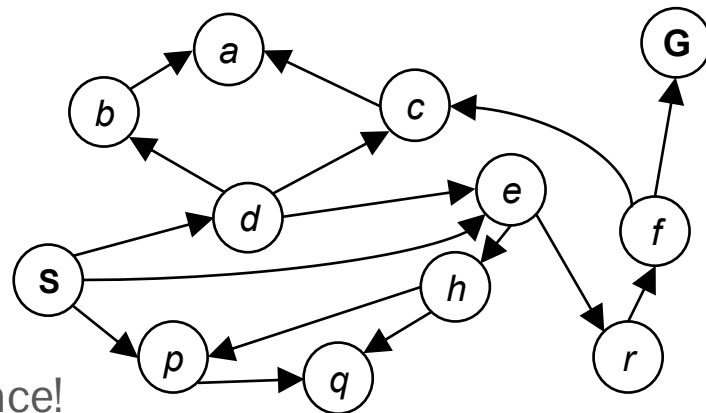
State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



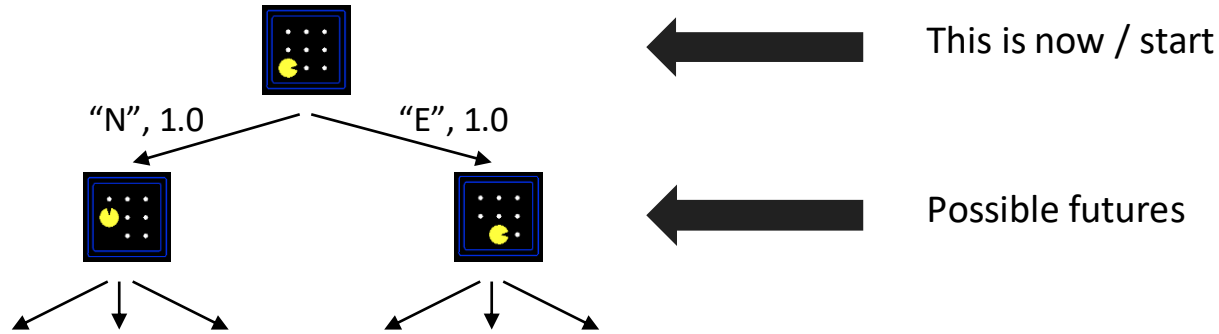
State Space Graphs

- State space graph: A mathematical representation of a search problem
 - Nodes are (abstracted) world configurations
 - Arcs represent successors (action results)
 - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



*Tiny search graph for a tiny
search problem*

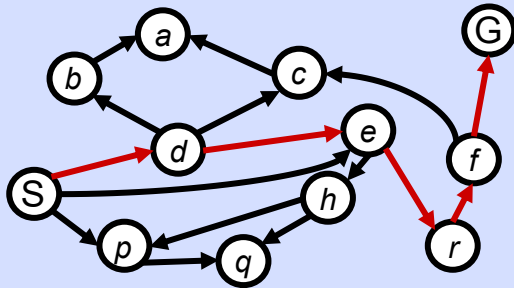
Search Trees



- A search tree:
 - A “what if” tree of plans and their outcomes
 - The start state is the root node
 - Children correspond to successors
 - Nodes show states, but correspond to PLANS that achieve those states
 - **For most problems, we can never actually build the whole tree**

State Space Graphs vs. Search Trees

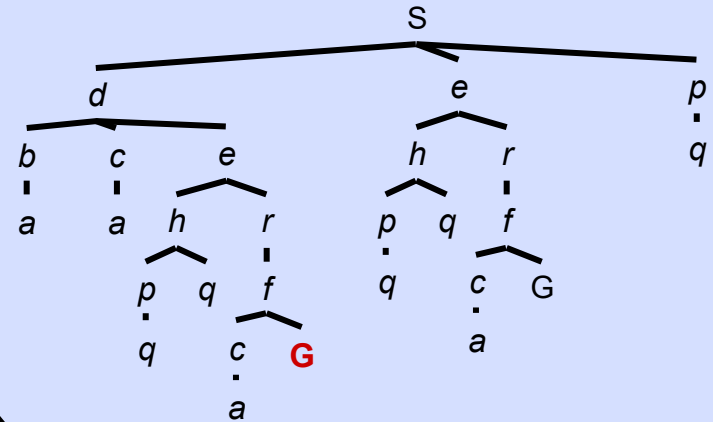
State Space Graph



Each NODE in in the search tree is an entire PATH in the state space graph.

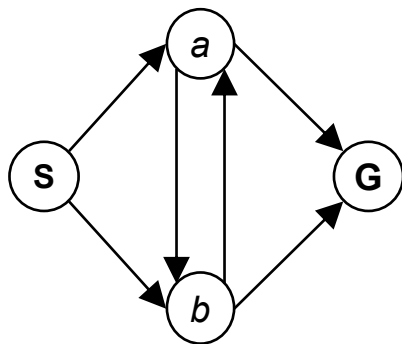
We construct both on demand – and we construct as little as possible.

Search Tree



Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:

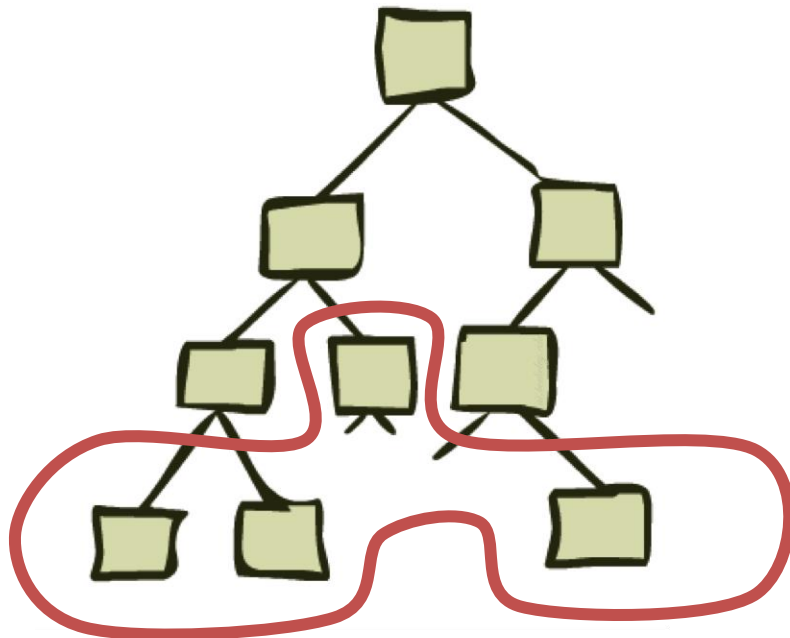


How big is its search tree (from S)?
assuming we are allowed to visit a
node multiple times.

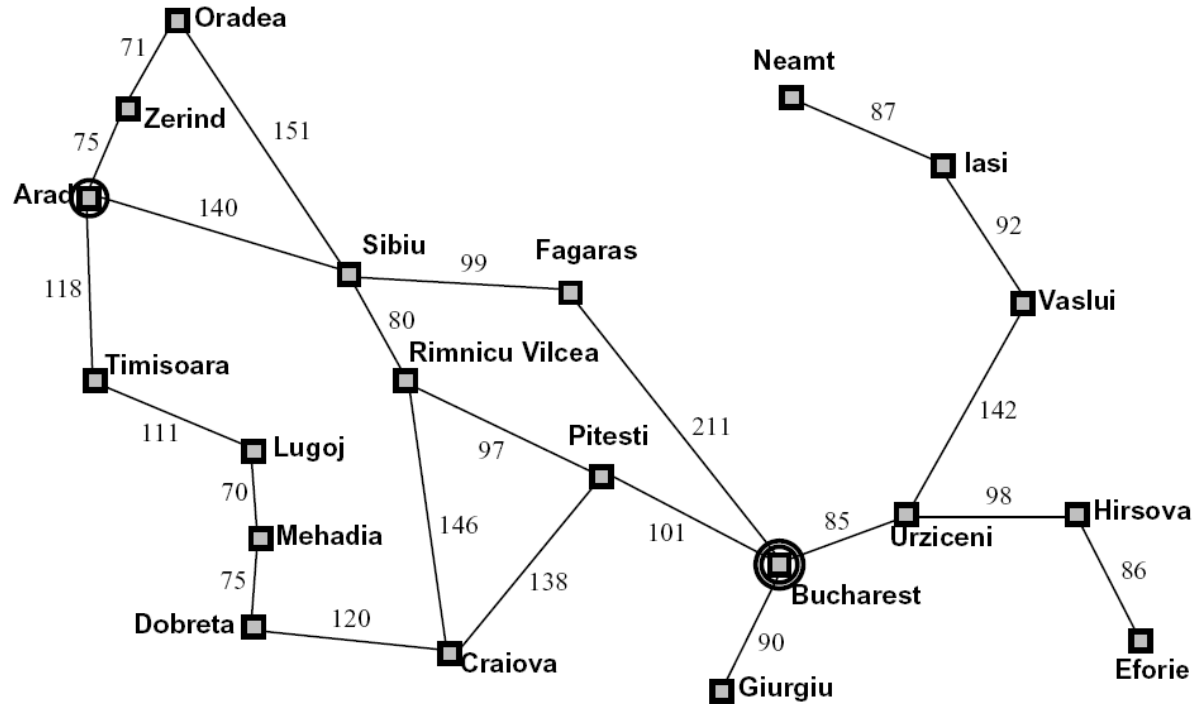


Important: Lots of repeated structure in the search tree!

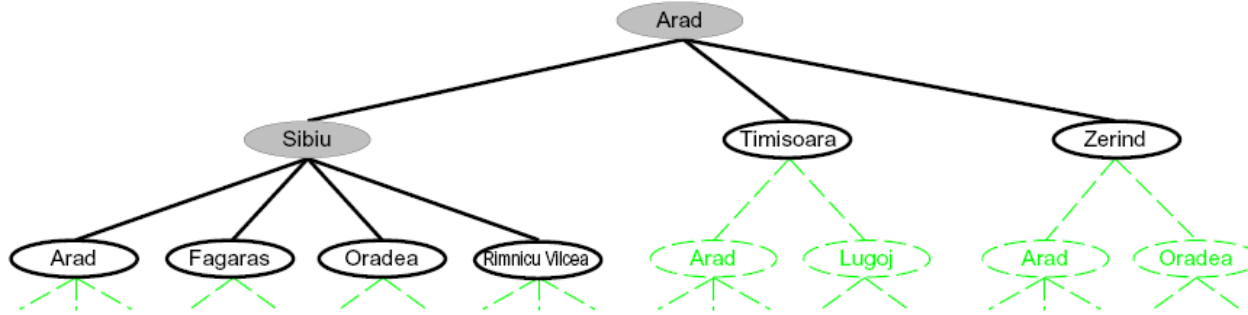
Tree Search



Search Example: Romania



Searching with a Search Tree



- Search:
 - Expand out potential plans (tree nodes)
 - Maintain a **fringe** of partial plans under consideration
 - Try to expand as few tree nodes as possible

General Tree Search

Attendance Code:

123

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

- Important ideas:
 - Fringe (usually implemented using queue or stack)
 - Expansion
 - Exploration strategy
- Main question: which fringe nodes to explore?

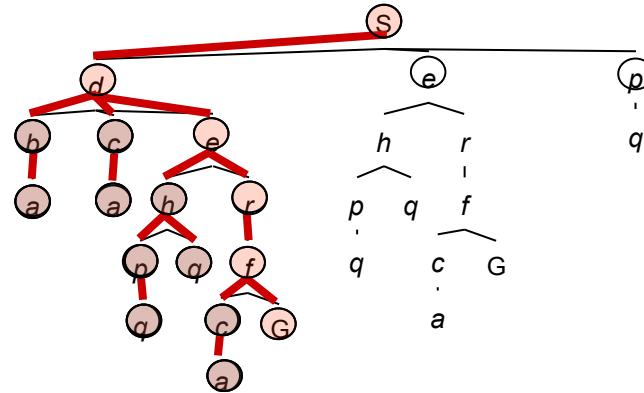
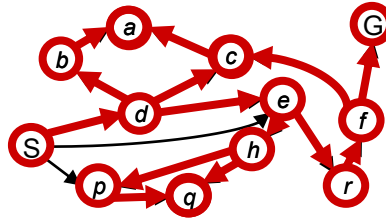
Depth-First Search



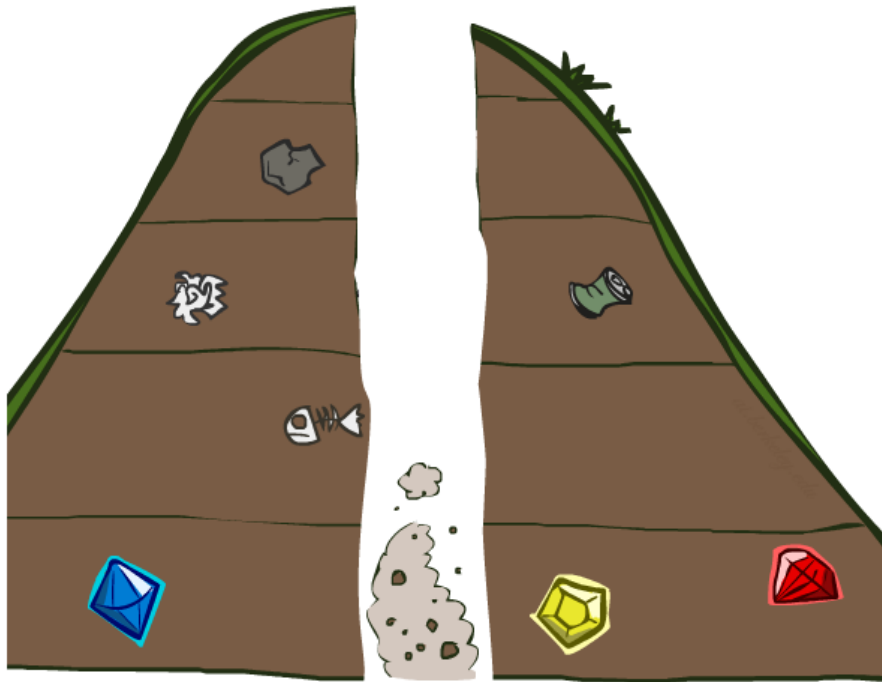
Depth-First Search

Strategy: expand a deepest node first

Implementation:
Fringe is a LIFO stack

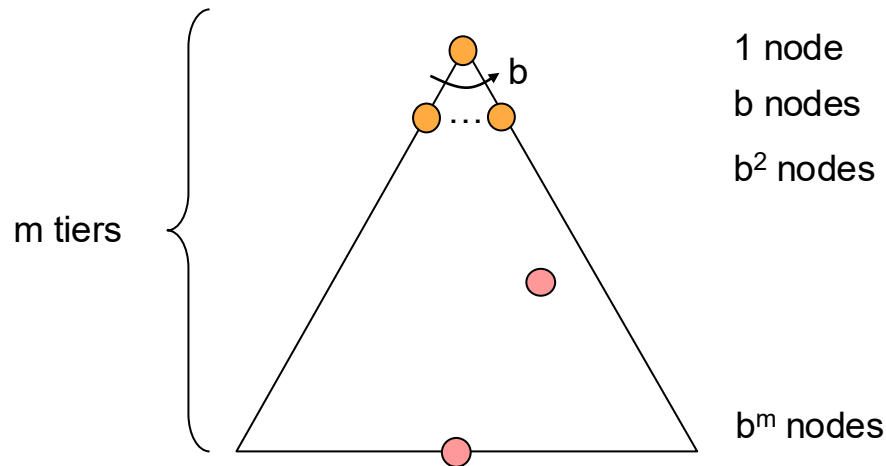


Search Algorithm Properties



Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity?
- Space complexity?
- Cartoon of search tree:
 - b is the branching factor
 - m is the maximum depth
 - solutions at various depths
- Number of nodes in entire tree?
 - $1 + b + b^2 + \dots + b^m = O(b^m)$



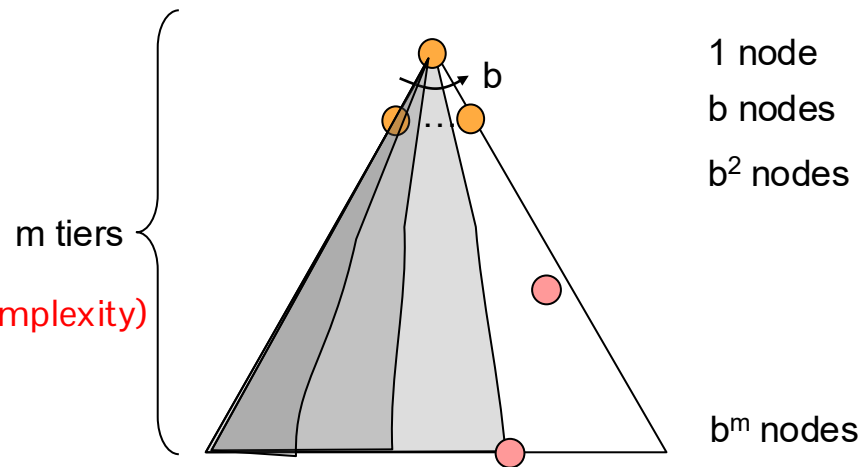
Big O Notation

- **Big O notation** is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity.
- Infinite asymptotics
- Big O notation is useful when analyzing algorithms for efficiency.
- The time (or the number of steps)
$$T(n) = 4n^2 - 2n + 2$$
- As n grows large, the n^2 term will come to dominate, so that all other terms can be neglected.

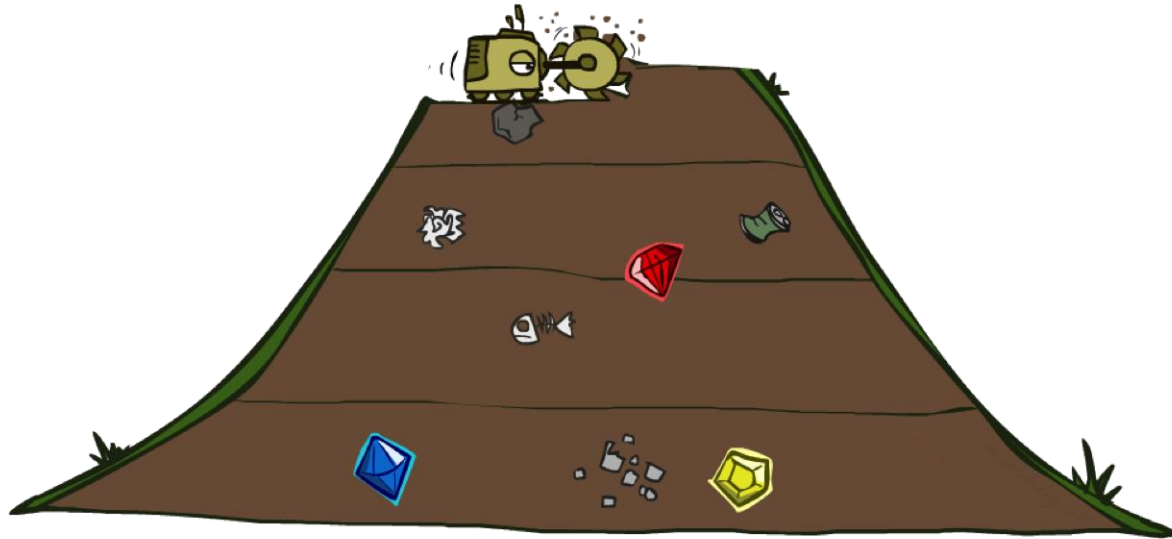
$$T(n) = O(n^2) \text{ or } T(n) \in O(n^2)$$

Depth-First Search (DFS) Properties

- What nodes DFS expand?
 - Some left prefix of the tree.
 - Could process the whole tree (Time complexity)
 - If m is finite, takes time $O(b^m)$
- How much space does the fringe take? (Space complexity)
 - Only has siblings on path to root, so $O(bm)$
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost



Breadth-First Search

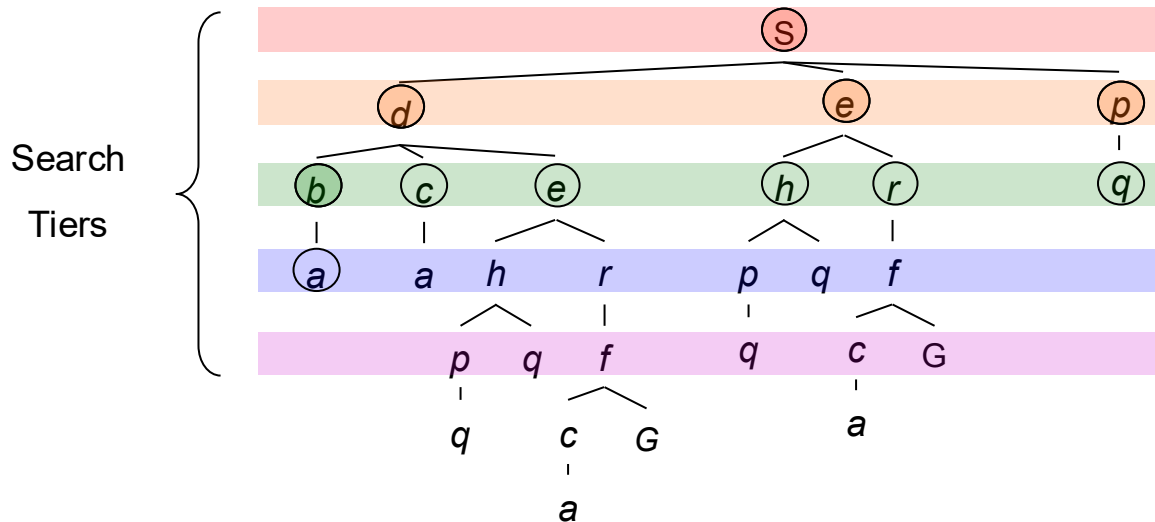
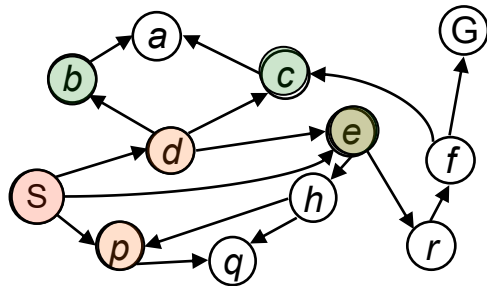


Breadth-First Search

Attendance Code:
123

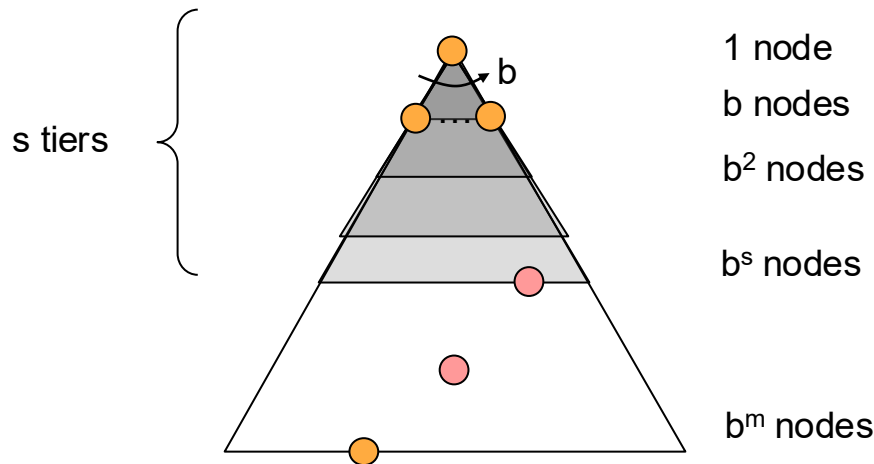
*Strategy: expand a
shallowest node first*

*Implementation: Fringe
is a FIFO queue*

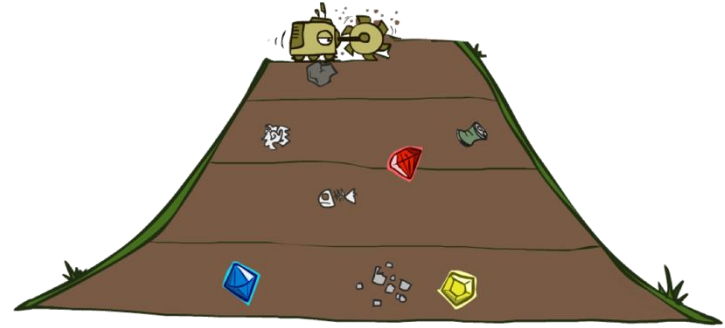


Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let **depth of shallowest solution** be s
 - Search takes time $O(b^s)$
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^s)$
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if **costs are all 1** (more on costs later)

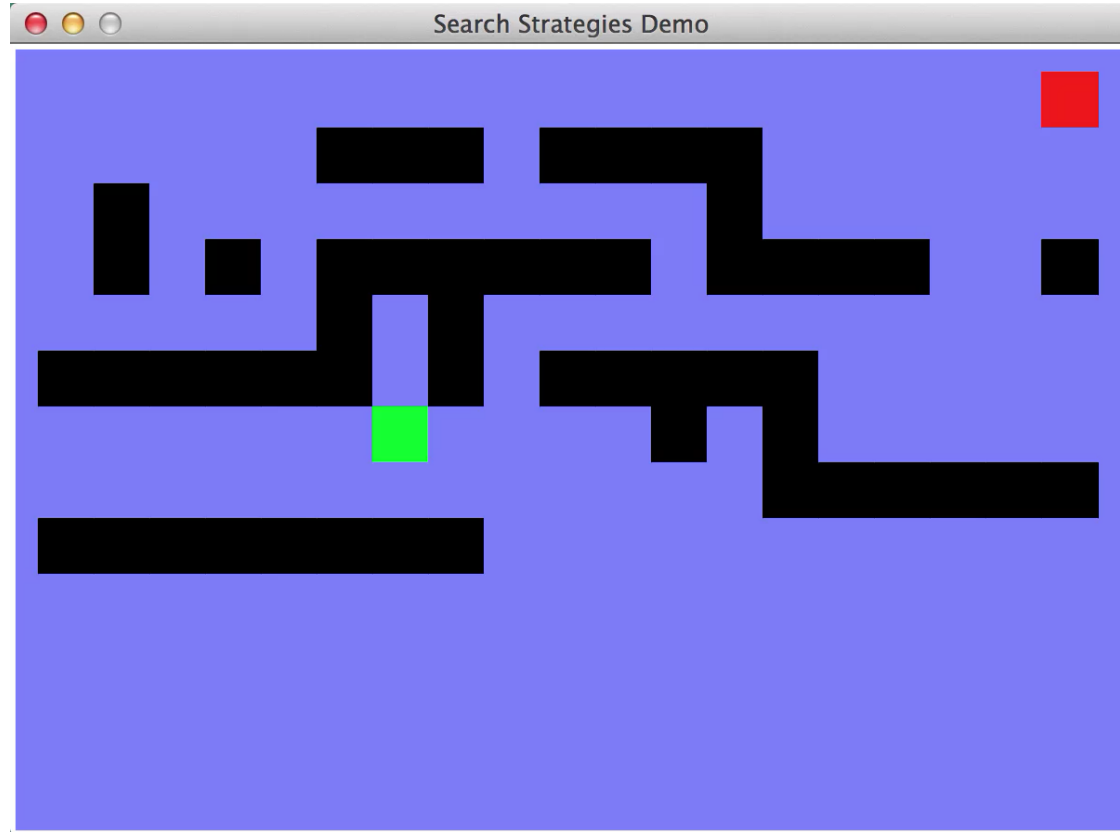


Quiz: DFS vs BFS

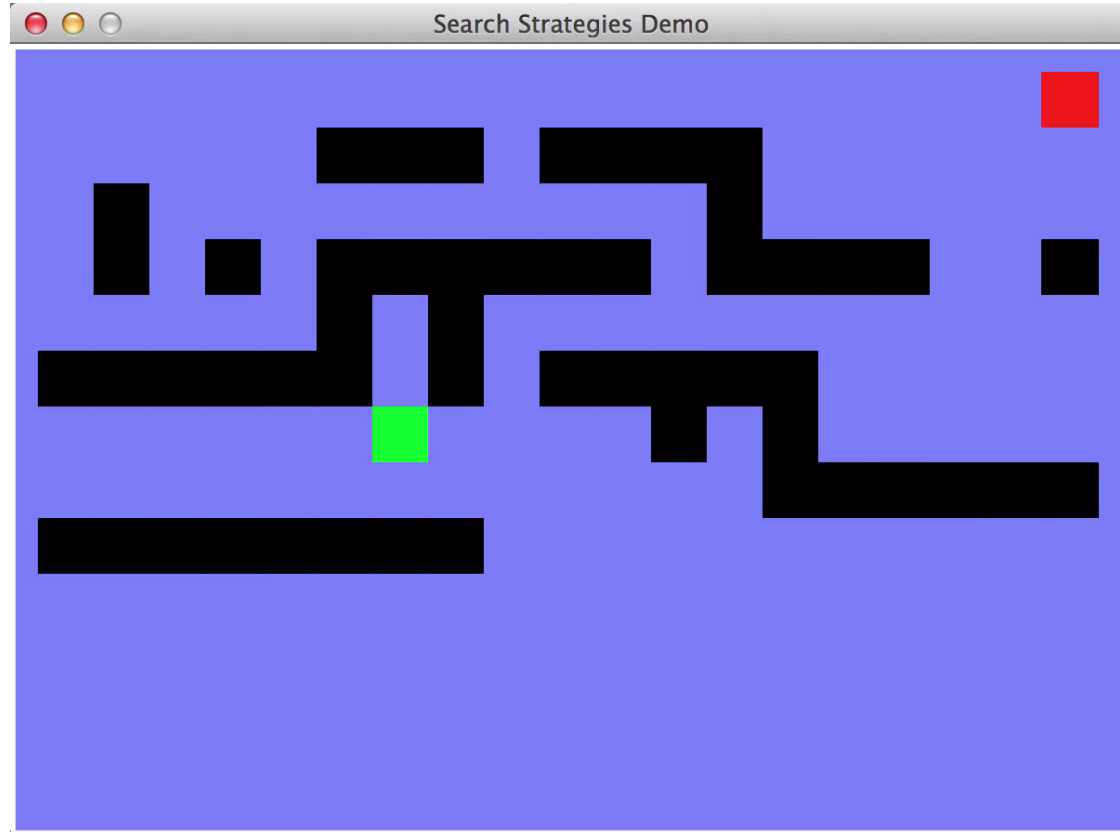


- Which algorithm is better?
- When does DFS outperform BFS? Or vice versa?

Video of Demo Maze Water DFS/BFS

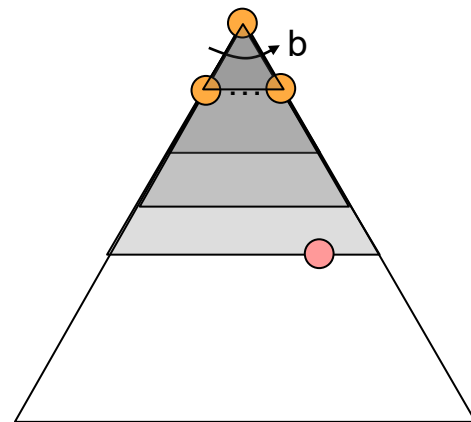


Video of Demo Maze Water DFS/BFS



Iterative Deepening

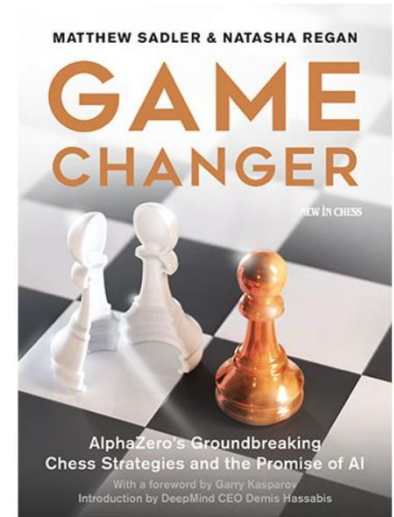
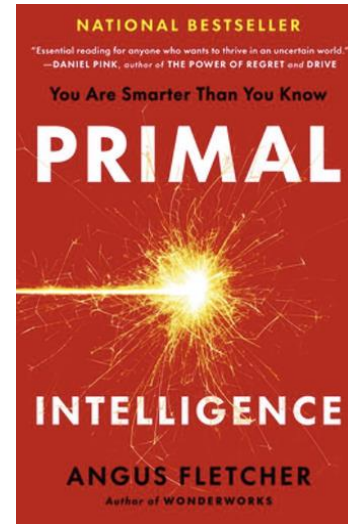
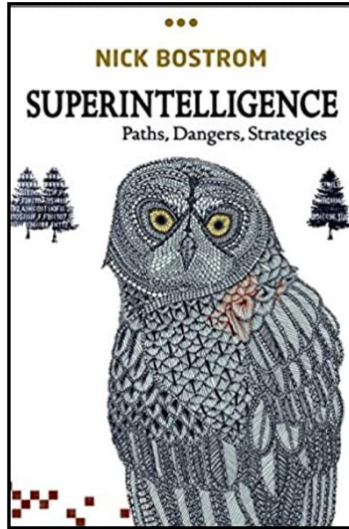
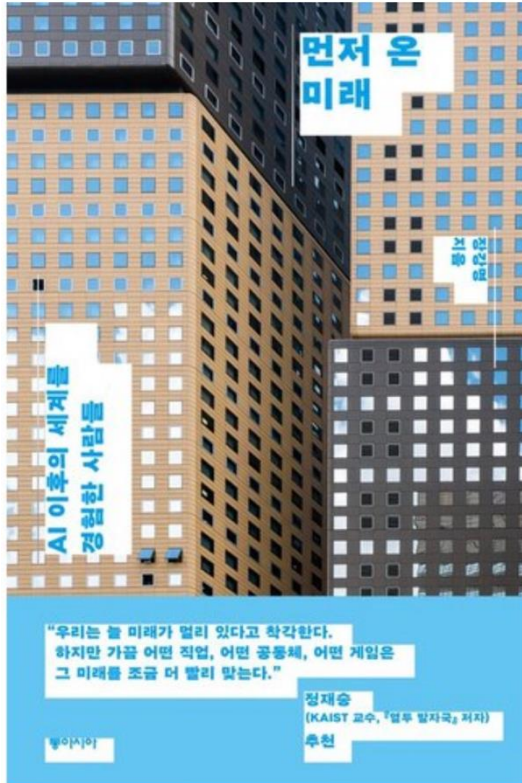
- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
 - Run a DFS with depth limit 1. If no solution...
 - Run a DFS with depth limit 2. If no solution...
 - Run a DFS with depth limit 3.
- Isn't that wastefully redundant?
 - Generally most work happens in the lowest level searched, so not so bad!



Questions

- In the same game, why does the size or complexity of the state space vary?
- What's the key difference between state space graphs and search trees?
- Why can a search problem be viewed as a model or simplified world?
- Between DFS and BFS, which one is a complete algorithm ? assuming the action cost is uniform.

Readings (HW 1: essay – book review)



What's next?

- Uninformed Search!
 - Uniform Cost Search